# 📚 Part 4: Mastering Collections in Dart

Your Simple & Complete Guide to Lists, Sets, and Maps

## 1. What is a Collection?

A **Collection** is a container that lets you store multiple items together inside a single variable box. Think of it like an organizing tray with multiple compartments.

| Feature | 📋 List | 💎 Set | 🗺️ Map |
|---|---|---|---|
| **Ordering** | ✅ **Ordered** (Items stay in exact sequence) | ❌ **Unordered** (Items have no fixed order) | ❌ **Unordered Pairs** |
| **Indexed** | ✅ **Yes** (Uses 0, 1, 2, 3...) | ❌ **No** (Cannot read via `[0]`) | ❌ **No** (Uses Keys instead) |
| **Duplicates** | ✅ **Allowed** (Can have identical values) | ❌ **Not Allowed** (Auto-removes repeats) | ❌ **Unique Keys** (Values can repeat) |
| **Syntax Layout** | Square Brackets: `[1, 2, 3]` | Curly Brackets: `{1, 2, 3}` | Key-Value Pairs: `{"key": "value"}` |

## 2. Deep Dive: 📋 List (Ordered Collection)

Lists keep your data strictly ordered. Every item is assigned an **Index** position starting from **0**.

```
void main() {
  List<String> fruits = ["Apple", "Banana", "Orange"];

  print(fruits[0]); // Output: Apple
  print(fruits.length); // Output: 3
}
```

### 🛠️ List Operations Cheat Sheet:

- `fruits.add("Grape");` → Adds an item at the very end.
- `fruits.insert(1, "Mango");` → Squeezes an item in at index position 1.
- `fruits.addAll(["Kiwi", "Melon"]);` → Glues multiple items to the end.
- `fruits.remove("Banana");` → Searches for and deletes that specific item value.
- `fruits.removeAt(0);` → Deletes whatever is sitting at index position 0.
- `fruits[0] = "Strawberry";` → Updates/replaces the item at index position 0.

### 💡 Storing Mixed Types (Heterogeneous Lists)

By default, a list restricts you to one type. To store different types together, use `dynamic` or omit the type completely:

```
List<dynamic> items1 = [1, "Hello", true]; // ✅ Allowed
var items2 = [2, "World", false];          // ✅ Allowed (Dart treats it as List<Object>)
```

## 3. Deep Dive: 💎 Set (Unique Collection)

A Set stores a group of items with **zero duplicates**. If you try to add an item that already exists, the Set simply ignores it.

```
void main() {
  Set<int> numbers = {1, 2, 2, 3, 3, 3};
  print(numbers); // Output: {1, 2, 3} (Duplicates automatically cleaned up!)

  // Set Math (Combining Sets)
  Set<int> primary = {1, 2, 3};
  Set<int> secondary = {3, 4, 5};
  primary.addAll(secondary);
  print(primary); // Output: {1, 2, 3, 4, 5}
}
```

## 4. Deep Dive: 🗺️ Map (Key-Value Pair Lookup)

Maps operate exactly like a dictionary or phonebook. Instead of counting numbers (0, 1, 2), you lookup values using a custom, unique **Key**.

```
void main() {
  // Syntax: Map<KeyType, ValueType>
  Map<String, int> userAges = {
    "Alice": 25,
    "Bob": 30
  };

  print(userAges["Alice"]); // Output: 25
  userAges["Charlie"] = 28;  // Adds a brand new entry
  userAges["Alice"] = 26;    // Updates Alice's age
}
```

> ### ⚡ Dynamic Maps & Complex Data Structural Access (Lists & Nesting)
>
> In real apps, values inside a map are often other collections, like a **List inside a Map** or a **Nested Map**. To handle this without type errors, use `dynamic` as the value type.

**A. Accessing a List Inside a Map**

Chain your map key lookup with a list index position:

```
Map<String, dynamic> studentData = {
  "name": "John",
  "hobbies": ["Coding", "Gaming", "Cooking"]
};

// Step 1: Look up the list key -> studentData["hobbies"]
// Step 2: Extract from index 0 -> [0]
print(studentData["hobbies"][0]); // Output: Coding
```

**B. Accessing a Nested Map (Map Inside a Map)**

Chain your map key lookups sequentially back-to-back:

```
Map<String, dynamic> company = {
  "manager": "Alice",
  "address": {
    "city": "Kathmandu",
    "country": "Nepal"
  }
};

// Step 1: Access the inner map key -> company["address"]
// Step 2: Access the specific inner key -> ["city"]
print(company["address"]["city"]); // Output: Kathmandu
```

## 5. Quick Property Reference Cheat Sheet

- `.length` → Counts how many items or key-value entries live inside the collection container.
- `.isEmpty` / `.isNotEmpty` → Checks if the container layout is totally blank or holds content.
- `.keys` / `.values` → (Maps Only) Extracts an iterable list view containing exclusively all structural keys or all data values.